

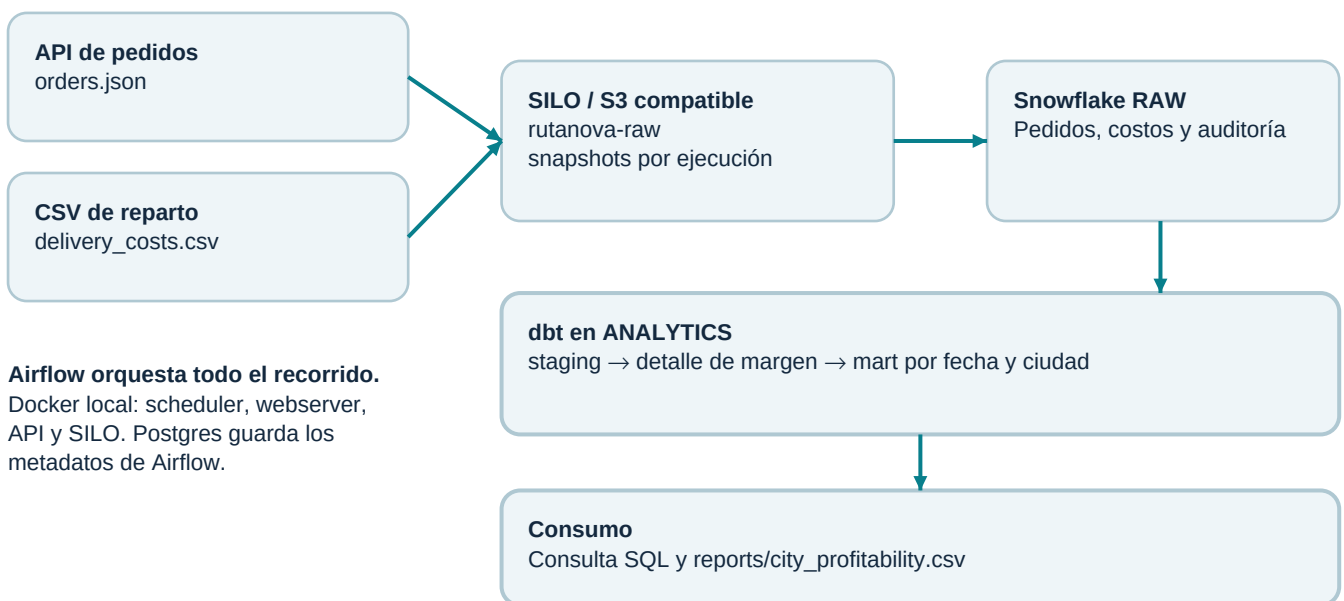
RutaNova

Rentabilidad diaria de pedidos entregados por ciudad

01 Caso de negocio

¿Qué ciudad deja más margen después del costo del producto y del reparto? RutaNova integra una API HTTP de pedidos y un CSV del transportista. Ambas fuentes son **simuladas y reproducibles**; el pipeline y Snowflake se ejecutan realmente. Se excluyen cancelados. El margen de contribución no incluye impuestos ni gastos fijos.

02 Arquitectura y recorrido del dato



03 Decisiones de diseño

Orquestación: DAG diario a las 07:00 de Lima, catchup=False, max_active_runs=1, TaskGroups y PythonSensor en reschedule. Retries con backoff, owner y doc_md.

Carga: snapshot completo con DELETE + INSERT en transacción; rollback ante fallos. Los originales quedan en SILO. DDL fuera de la transacción; sin CDC.

Modelado: 4 modelos encadenados. Margen = ingreso - producto - reparto; porcentaje ponderado por ingreso. dbt build se lanza desde PythonOperator (sin Cosmos).

Seguridad: Connections cifradas en Airflow, profiles.yml con env_var(), clave privada fuera de Git e imagen. Usuario RUTANOVA_SVC y rol limitado RUTANOVA_ROLE; XSMALL con AUTO_SUSPEND=60.

RutaNova

Validaciones, ajustes y entrega reproducible

04 Evidencia de ejecución real

Airflow: Todas las tareas de la ejecución mostrada en success.

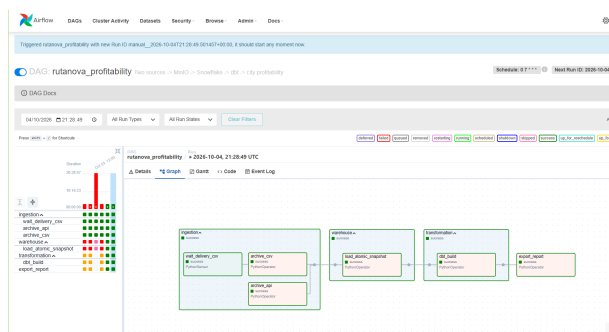
Almacenamiento: Bucket privado; orders.json y delivery_costs.csv por snapshot.

pytest: 15 passed in 2.03s; reglas de negocio y estructura del DAG.

dbt: 4 modelos + 20 tests; PASS=24, WARN=0, ERROR=0, SKIP=0.

Snowflake: Mart visible con 9 filas: 3 fechas por Lima, Arequipa y Trujillo.

Fixture: 18 pedidos, 15 entregados y 3 cancelados. Totales de referencia: **SI 2,850 de ingresos y SI 779 de margen**; verificar con sql/02_validate_results.sql. Las fechas del ejemplo son 01-03/10/2026.



Airflow: tareas completas en success

The screenshot shows the Snowflake query results for the query 'SELECT COUNT(*) AS ORDERS FROM RAW.ORDERS;'. The results are displayed in a table with 9 rows and 1 column. The table has a header row with columns: ID, ORDER_DATE, CITY, DELIVERED_ORDERS, TOTAL_REVENUE, TOTAL_PRODUCT_COST, TOTAL_DELIVERY_COST, and CONTRIB.

ID	ORDER_DATE	CITY	DELIVERED_ORDERS	TOTAL_REVENUE	TOTAL_PRODUCT_COST	TOTAL_DELIVERY_COST	CONTRIB
1	2026-10-01	Arequipa	2	270.00	160.00	30.00	
2	2026-10-01	Lima	2	200.00	100.00	20.00	
3	2026-10-01	Trujillo	1	100.00	50.00	10.00	
4	2026-10-02	Arequipa	2	300.00	200.00	30.00	
5	2026-10-02	Lima	2	370.00	230.00	34.00	
6	2026-10-03	Trujillo	1	100.00	100.00	14.00	
7	2026-10-03	Arequipa	2	500.00	300.00	30.00	
8	2026-10-03	Lima	2	400.00	320.00	26.00	
9	2026-10-03	Trujillo	1	200.00	100.00	18.00	

Snowflake: mart por fecha y ciudad

05 Ajustes incorporados

Docker: pgsty/minio:RELEASE.2026-08-04T00-00-00Z (SILO) y clickhouse/minio-mc:RELEASE.2025-04-16T18-13-26Z. Airflow en localhost:8081; consola S3 en localhost:9001.

Snowflake: registro de la clave pública completa correspondiente a la privada del scheduler; creación de warehouse, database, schemas y permisos. Sesión comprobada con usuario, rol, base y warehouse correctos.

Pruebas: COPY de api/ e inbox/ al Dockerfile; reconstrucción y recreación de servicios. Corrige el FileNotFoundError de los datos de prueba.

06 Reproducibilidad y alcance

README: configurar .env local, construir Docker, generar el par de claves, ejecutar DDL/grants en Snowflake, levantar Compose, disparar el DAG y validar resultados. El CI definido ejecuta business-tests y docker-dag-tests; parsea dbt offline. La protección de main y el PR verde deben verificarse en GitHub antes de entregar. Despliegue manual con Compose, sin CD automático ni alta disponibilidad.

07 Enlaces de entrega

Completar ambos campos con las URLs reales antes de entregar. Video de 5-8 minutos; repositorio público con PR y CI aprobados.

Repositorio público

Video demo